mbusgw-t2r

User Guide

Document Number: DO-T2RUG-EN-01A
Publication Date: August 2026
Revision Update Information: This is a new manual.
Operating System: Linux (glibc), kernel 4.x or later
Software Version: mbusgw-t2r X.00-09

NaPi World & StarLet Squad collaboration

mbusgw-t2r User Guide

Copyright © 2026 NaPi World & StarLet Squad collaboration

Legal Notice

The information contained herein is subject to change without notice. NaPi World & StarLet Squad collaboration shall not be liable for technical or editorial errors or omissions contained herein.

The software described in this manual is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

MODBUS is a registered trademark of Schneider Electric USA, Inc. Linux is a registered trademark of Linus Torvalds. UNIX is a registered trademark of The Open Group.

Contents

Preface iv

1. About This Manual iv
2. Intended Audience iv
3. Document Structure iv
4. Related Documents iv
5. Conventions iv

Chapter 1. What the Gateway Does 1

Chapter 2. Before You Start 2

Chapter 3. The Settings File 3

3.1. The serials Section 3
3.2. The listeners Section 3

Chapter 4. Running the Gateway 5

4.1. The First Test with mbpoll 5

Chapter 5. The Time Stamp Pseudo Device 6

Chapter 6. Reading the Log 7

Chapter 7. Troubleshooting 8

7.1. The Symptom, the Cause, the Action 8
7.2. The Message Codes Reference 8
7.3. If Nothing Helps 9

Preface

1. About This Manual

This manual describes the configuration, the operation and the troubleshooting of mbusgw-t2r --- a gateway which translates MODBUS TCP requests coming from the network into MODBUS RTU transactions on a serial (RS-485 or RS-232) line, and returns the answers back.

2. Intended Audience

This manual is written for a reader with no prior MODBUS or Linux administration experience: every step is spelled out and nothing is assumed. An experienced engineer may go straight to Chapter 3 (the settings file) and Chapter 7 (the troubleshooting).

3. Document Structure

Chapter 1 explains what the gateway does. Chapter 2 is the pre-flight checklist. Chapter 3 describes the settings file key by key. Chapter 4 covers the start, the command line options and the first end-to-end test. Chapter 5 describes the built-in Time Stamp pseudo device. Chapter 6 teaches how to read the log. Chapter 7 is the troubleshooting reference: the symptom, the cause, the action, and the reference of all the message codes.

4. Related Documents

README.md --- the installation sequence (the StarLet package, the build, `make install`). MODBUS over Serial Line Specification and Implementation Guide and MODBUS Application Protocol Specification --- the protocol itself, available from modbus.org.

5. Conventions

Convention	Meaning
`Monospace type`	Code examples, file names, commands and interactive screen displays.
`%T2R-E-CODE`	A message code as it appears in the gateway log; see Section 7.2.
`<...>`	A placeholder to be replaced by an actual value, e.g. a device name.
`$`	The shell prompt of an unprivileged user; do not type it.
`Ctrl/C`	Hold down the key labeled Ctrl while you press the key C.

Chapter 1. What the Gateway Does

You have a device (an electricity meter, a temperature controller, a PLC ...) which talks MODBUS RTU over a serial line. And you have a program (a SCADA, a script, `mbpoll` ...) which talks MODBUS TCP over the network. The gateway sits between them and translates, as Figure 1 shows.

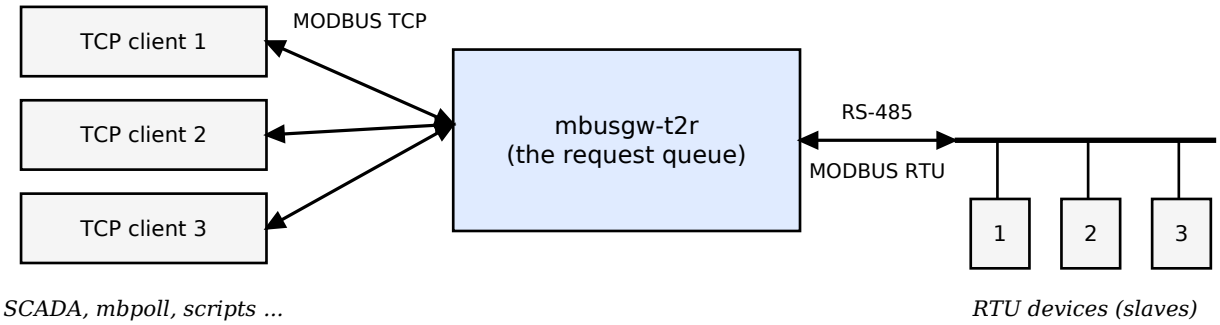


Figure 1. The data path of the gateway

Several TCP clients may connect at the same time: the gateway queues them so that only one request at a time goes to the serial line --- an RTU line physically cannot carry two transactions at once.

Several devices may share one RS-485 line. They are told apart by the slave address inside the MODBUS request itself; the gateway does not need to know their addresses at all.

Chapter 2. Before You Start

Walk this checklist once --- it saves an hour of guessing later.

1. The gateway is installed (see README.md). After the installation you have the program `/opt/mbusgw-t2r/sbin/mbusgw-t2r` and the settings file `/opt/mbusgw-t2r/etc/modbus-t2r_settings.conf`.

2. You know which serial port the device is connected to. On Linux it is a file like `/dev/ttyS0` or `/dev/ttyUSB0`. If unsure, plug the USB adapter out and in, then run:

```
$ dmesg | tail
```

3. You know the line parameters of the device: the speed, the data bits, the parity and the stop bits. They are in the device manual; the most common set is `9600, 8, N, 1`.

4. Your user may open the port. Check the group of the port file:

```
$ ls -l /dev/ttyUSB0
crw-rw---- 1 root dialout 188, 0 ... /dev/ttyUSB0
```

If the group is `dialout`, add yourself to it and re-login (or simply run the gateway with `sudo`):

```
$ sudo usermod -a -G dialout $USER
```

Chapter 3. The Settings File

The settings file has two sections: `serials` (the serial lines) and `listeners` (the TCP ports). A minimal working example:

```
serials = (
{ device = "/dev/ttyUSB0";
  chars  = "9600, 8, N, 1";
  iotmo  = 1000;
}
);

listeners = (
{ bind   = "TCP:0.0.0.0:502";
  target = "/dev/ttyUSB0";
  connlm = 8;
}
);
```

Read it as: "open the port /dev/ttyUSB0 at 9600-8-N-1 and wait for an answer up to 1000 ms; listen for TCP clients on every interface, port 502, and forward them to that port".

3.1. The serials Section

One record per serial port. Table 3-1 lists every key.

Key	Required	Meaning and the allowed values
`device`	yes	The serial port file, e.g. /dev/ttyUSB0.
`chars`	yes	The line parameters: speed, data bits, parity, stop bits. Speed 50..4000000; data bits 5..8; parity N (none), E (even), O (odd); stop bits 1..2.
`iotmo`	no	How long to wait for the answer of the device, milliseconds. Default 1000.
`rs485`	no	1 --- ask the kernel to drive the RS-485 direction control (only for the ports which support it). Default 0.
`ts_enabled`	no	1 --- enable the Time Stamp pseudo device (Chapter 5). Default 0.
`ts_unit`	no	The slave address the pseudo device answers on. Default 135.
`ts_fncode`	no	The function code it answers to. Default 4.
`ts_base_reg0`	no	The first register of the time stamp. Default 135.

If a record is wrong, the gateway skips it and says why --- with the allowed range right in the message:

```
%T2R-E:  [serial #00:</dev/ttyUSB0>] --- speed 31 baud is out of range [50..4000000]
```

3.2. The listeners Section

Key	Required	Meaning and the allowed values
`bind`	yes	Where to listen: TCP:<IP address>:<port>. The port is 1..65535; 0.0.0.0 means every interface. UDP is not supported.
`target`	yes	Which serial port to forward to. Must match a device from serials exactly, character by character.

Key	Required	Meaning and the allowed values
`connlm`	no	How many TCP clients may wait in the connection queue, e.g. 8.
`iotmo`	no	The network I/O timeout, milliseconds.

Port 502 is the standard MODBUS TCP port, but on Linux the ports below 1024 need the root privileges. If you do not want to run as root, take e.g. `5502` and point your client there. Several listeners may share one target: the gateway serializes their requests.

Chapter 4. Running the Gateway

By hand, in a terminal (the messages go straight to the screen):

```
$ /opt/mbusgw-t2r/sbin/mbusgw-t2r /settings=/opt/mbusgw-t2r/etc/modbus-t2r_settings.conf
```

Option	Meaning
`/settings=<file>`	The settings file (Chapter 3).
`/trace`	The verbose tracing: every request and answer is dumped in hex. Priceless at the first run.
`/logfile=<file>`	Write the log to a file instead of the screen.
`/logsize=<octets>`	Rotate the log file above this size.

A healthy start prints (shortened):

```
%T2R-I-REVISNF, Rev: T2R X.00-06/aarch64(built at ...) (REV: 00.06.00)
%T2R-I:  Added device #00 [</dev/ttyUSB0>, Chars: <9600, 8, N, 1>, ...] --- added
%T2R-I:  Added listener #00 [Target: </dev/ttyUSB0>, Net: <TCP:0.0.0.0:502>, ...] --- added
%T2R-S-DEVREADY, Device </dev/ttyUSB0> [9600 baud, answer tmo: 1000 msec, t3.5: 4010 usec] --- is r
%T2R-S-LSNRRDY, [#3] Listener 0.0.0.0:502 [Target: </dev/ttyUSB0>] --- is ready
```

Two lines matter most: DEVREADY (the port is open) and LSNRRDY (the TCP port is listening). If you see both, the gateway is up. Figure 2 shows what happens to every request after that.

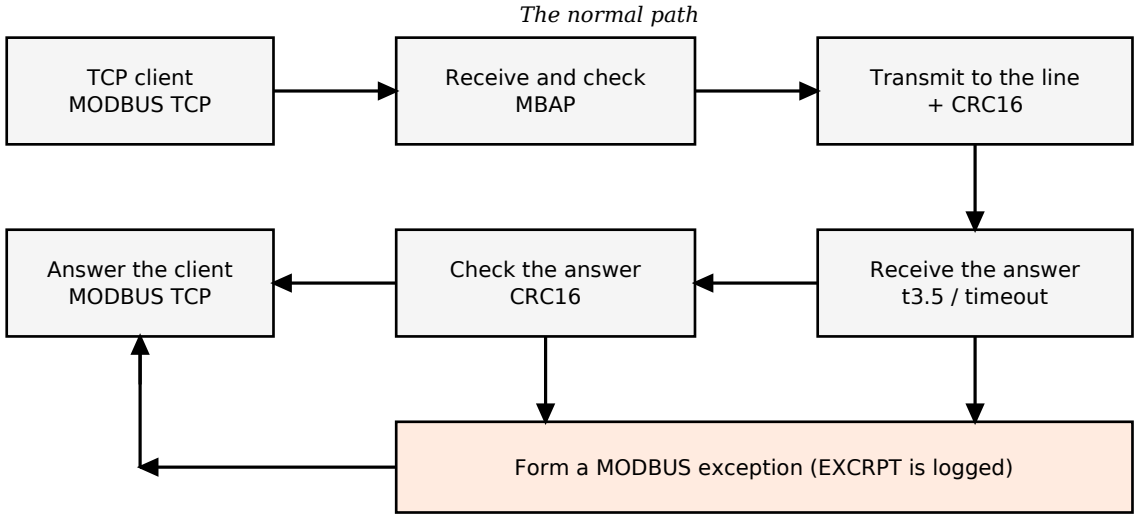


Figure 2. The lifecycle of one request

To stop the gateway press Ctrl/C once and wait a second. To toggle the tracing of a running gateway without a restart:

```
$ kill -USR1 <pid>
```

4.1. The First Test with mbpoll

`mbpoll` is a small free MODBUS client (`sudo apt install mbpoll`). Read 3 holding registers from the device with the slave address 1:

```
$ mbpoll -a 1 -r 1 -c 3 -1 127.0.0.1 -p 502
```

If the numbers come back --- everything works, end to end.

Chapter 5. The Time Stamp Pseudo Device

The gateway can answer by itself --- without touching the line --- to a read request sent to a special slave address. The answer is the current time of the gateway host. This lets a SCADA read the precise time over the plain MODBUS.

With `ts_enabled = 1; ts_unit = 135; ts_fncode = 4; ts_base_reg0 = 135;` a request "read N input registers starting at 135 from the slave 135" returns the registers of Figure 3.

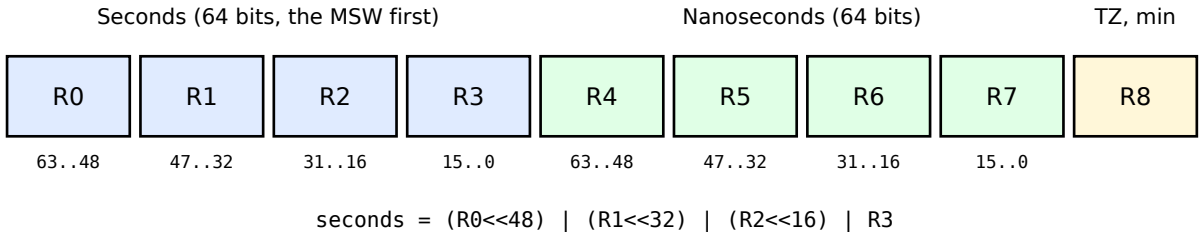


Figure 3. The Time Stamp registers layout

Registers requested	You get
4	The seconds of the UNIX epoch, 64 bits, the most significant 16-bit word first.
8	Plus the nanoseconds, packed the same way into R4..R7.
9	Plus R8 = the local timezone offset, in signed minutes (e.g. Moscow = +180).

Any other register count in a TS request is answered with the MODBUS exception ILLEGAL_DATA_ADDRESS --- this is on purpose.

Chapter 6. Reading the Log

Every important event is one line with a code:

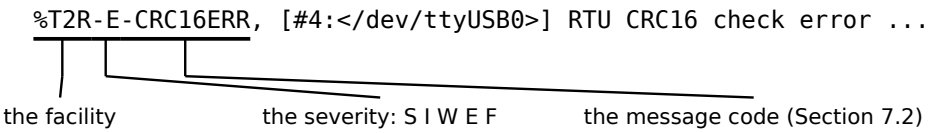


Figure 4. The anatomy of a log message

The severity letter is: S --- success, I --- information, W --- warning, E --- error, F --- fatal. The part `[#4:</dev/ttyUSB0>]` means: the file descriptor #4, this serial port.

Grep the log by the code --- this is exactly why the codes exist:

```
$ grep CRC16ERR /var/log/mbusgw-t2r.log
```

Chapter 7. Troubleshooting

The golden rule: run with /trace and read the log. The gateway always says what it dislikes.

7.1. The Symptom, the Cause, the Action

You see	It means / what to do
`%T2R-E-DEVOPNERR, ... errno: 2`	The port file does not exist: the adapter is unplugged or the name is wrong. Run dmesg | tail after plugging; fix device in the settings.
`%T2R-E-DEVOPNERR, ... errno: 13`	Permission denied. Run with sudo, or add yourself to the dialout group.
`%T2R-E-DEVOPNERR, ... errno: 16`	The port is busy. Find the holder: sudo fuser /dev/ttyUSB0, and stop it.
`%T2R-E-NOANSWER, ...`	The request went out, nothing came back: 1) a wrong slave address in the client request; 2) the wiring (swap A/B of RS-485); 3) a wrong speed/parity --- re-check chars; 4) the device is off.
`%T2R-E-CRC16ERR, ...`	The octets arrive damaged: 1) chars does not match the device (the most common!); 2) the line noise --- the grounding, the termination, the cable length; 3) two masters on one line.
`%T2R-E-BADFRAME, ...`	Something arrived, but it is not a sane MODBUS frame: usually the same causes as CRC16ERR; 400+ octets of garbage mean a foreign, non-MODBUS device on the line.
`%T2R-W-FRAMETMO, ...`	The answer started but never finished: a very slow or hanging device. Raise iotmo; check the cable.
`%T2R-W-EXCRPT, ... (SERVER_DEVICE_FAILURE)`	The gateway told the client "the serial side failed". The real reason is in the previous log line (NOANSWER, CRC16ERR ...).
`%T2R-E-LSNRERR, ... errno: 98`	The TCP port is taken: a second gateway instance, or another program. Find it: sudo ss -tlnp | grep 502.
`%T2R-E-LSNRERR, ... errno: 13`	The ports below 1024 need root. Run with sudo, or take a port above 1024 (e.g. 5502).
`... out of range [a..b]`	A settings value is out of its range; the allowed range is right in the message. Fix the value.
`No serials has been defined!`	Not a single serials record survived the validation. Read the error lines above --- each rejected record says why.
`The data is wrong or shifted`	The byte order confusion on the client side: the MODBUS registers are the big-endian 16-bit words; check how the client assembles the 32/64-bit values.

7.2. The Message Codes Reference

Code	Sev	When it appears
`REVISNF`	I	At the start: the program version. Quote it when asking for help.
`DEVREADY`	S	The serial port is open; the actual baud, the answer timeout and the t3.5 interval are shown.
`LSNRRDY`	S	The TCP port is listening; the address, the port and the target device are shown.

Code	Sev	When it appears
NETCONN	S	A TCP client has connected; its address:port and the listener are shown.
NETDISCN	S	A TCP client has disconnected; its address:port are shown.
DEVOPNERR	E	The serial port cannot be opened; errno says why (2 = no such file, 13 = the permissions, 16 = busy).
LSNRERR	E	bind()/listen() failed for a TCP port; errno says why (98 = taken, 13 = the privileges).
NOANSWER	E	The device did not answer within the iotmo timeout.
FRAMETMO	W	The answer began but was not completed within the timeout.
BADFRAME	E	The received frame length is outside the sane [5..256] octets range.
CRC16ERR	E	The checksum of the received frame does not match; both values are shown.
EXCRPT	W	A MODBUS exception is returned to the TCP client; the code and its name are shown.
EXITST	I	The gateway exits; the exit flag and the final status are shown.

7.3. If Nothing Helps

Collect and attach to your question: 1) the full start-up log with /trace (from REVISNF to the first error); 2) your settings file; 3) the output of `ls -l <device>` and `dmesg | tail -20`; 4) the exact model of the RTU device and its documented line parameters. With these four things the problem is almost always visible at a glance.